

Parameterized Quantum Circuits for Binary Classification

Iris classification with parameter-shift training, finite-shot tests, and robustness checks

Marius Torsheim

Department of Physics, University of Oslo

Spring 2026

Abstract

I implemented a quantum machine-learning classifier for the binary Iris data set using a transparent NumPy state-vector simulator. Each sample is encoded into a four-qubit state, processed by a trainable parameterized circuit, and classified from the probability of measuring the final qubit in the state $|1\rangle$. Parameters were trained with full-batch Adam and analytical parameter-shift gradients. The basic H - R_z encoder with a simple R_y -CNOT ansatz reached 0.84 test accuracy on the selected Iris split, while logistic regression reached 1.00. A circuit-variation study showed that the result depends strongly on feature map and ansatz: the R_y /simple and $R_y R_z$ /strong quantum circuits reached 1.00 test accuracy on the same split. To address the sensitivity of this small data set, I added a repeated-seed study: the mean test accuracies were approximately 0.804 ± 0.053 , 0.992 ± 0.019 , and 0.956 ± 0.045 for H - R_z /simple, R_y /simple, and $R_y R_z$ /strong circuits, respectively, compared with 1.000 ± 0.001 for logistic regression. A finite-shot measurement study showed that 100 shots mainly affects the weaker basic circuit, while 1000 and 10000 shots reproduce the exact-probability conclusions. As an optional extension, the stronger quantum model reached 0.874 test accuracy on the first four Breast Cancer features, close to logistic regression at 0.881. These results show that small quantum circuits can learn useful classifiers and that circuit design matters, but they do not demonstrate quantum advantage for these small classical data sets.

Repository: https://github.com/Torsheim/FYS5419_Project_2

I Introduction

Quantum machine learning uses quantum states and parameterized quantum circuits as trainable models for data analysis. The motivation is that an n -qubit state is described by 2^n complex amplitudes. This large Hilbert space does not automatically imply a practical advantage, because the circuits available in near-term calculations are shallow and highly constrained. A more useful question is therefore whether a small circuit can learn a useful classifier, how sensitive the result is to the chosen feature map and ansatz, and how it compares with a simple classical baseline.

This project studies Alternative 2 of FYS5419 Project 2: quantum machine learning on the first two targets of the Iris data set. The required tasks are to encode classical features into quantum states, add parameterized gates, measure a qubit to obtain predictions, build a model for many samples, train it with the parameter-shift rule, compare with logistic regression, and test circuit variations. I also include the optional Breast Cancer experiment suggested in the project text.

The report follows the same scientific structure as the previous project report: theory and algorithms are presented before results, numerical values are summarized in tables, figures are interpreted in the text, and reproducibility information is included at the end. Section II defines the circuit model, feature maps, loss function, and parameter-shift gradients. Section III describes the code and validation tests. Section IV presents the Iris results, circuit variations, repeated-seed robustness, finite-shot measurements, and the optional Breast Cancer extension. Section V summarizes the conclusions and limitations.

II Theory

A State-vector circuit model

The circuit is simulated as a pure state vector

$$|\psi\rangle = \sum_{z=0}^{2^n-1} c_z |z\rangle, \quad \sum_z |c_z|^2 = 1, \quad (1)$$

where n is the number of qubits. The initial state is $|0\rangle^{\otimes n}$. Single-qubit gates and CNOTs are applied directly to the state vector. For example,

$$R_y(\alpha) = \begin{pmatrix} \cos(\alpha/2) & -\sin(\alpha/2) \\ \sin(\alpha/2) & \cos(\alpha/2) \end{pmatrix}, \quad (2)$$

and

$$R_z(\alpha) = \begin{pmatrix} e^{-i\alpha/2} & 0 \\ 0 & e^{i\alpha/2} \end{pmatrix}. \quad (3)$$

The code uses one consistent little-endian convention for indexing basis states. This affects only how state-vector indices are mapped to qubit labels, not the physical probabilities.

B Feature maps and trainable circuits

For a scaled input vector $\mathbf{x} = (x_1, \dots, x_n)$, the basic feature map is

$$|0\rangle \mapsto R_z(2\pi x_j) H |0\rangle \quad (4)$$

for each qubit j . I call this the `h_rz` encoder. The variation study also uses

$$\text{ry} : |0\rangle \mapsto R_y(2\pi x_j) |0\rangle, \quad (5)$$

$$\text{ry_rz} : |0\rangle \mapsto R_z(2\pi x_j) R_y(\pi x_j) H |0\rangle. \quad (6)$$

The simple ansatz repeats R_y rotations and nearest-neighbour CNOT chains. With L layers and n qubits it contains

$$N_{\theta}^{\text{simple}} = 2Ln \quad (7)$$

trainable parameters. The stronger ansatz applies R_y and R_z rotations, a CNOT ring, and R_x rotations, giving

$$N_{\theta}^{\text{strong}} = 3Ln. \quad (8)$$

For the reported four-feature, two-layer experiments, the simple and strong circuits have 16 and 24 parameters respectively.

C Prediction, loss, and parameter shift

The classifier output is the probability of measuring the selected qubit in state $|1\rangle$:

$$f(\mathbf{x}; \boldsymbol{\theta}) = P(q_m = 1 \mid \mathbf{x}, \boldsymbol{\theta}) = \sum_{z: z_m=1} |c_z(\mathbf{x}, \boldsymbol{\theta})|^2. \quad (9)$$

The predicted class is $\hat{y} = 1$ if $f \geq 0.5$. The loss function is the mean binary cross-entropy,

$$L(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log f_i + (1 - y_i) \log(1 - f_i)], \quad (10)$$

where $f_i = f(\mathbf{x}_i; \boldsymbol{\theta})$. Small numerical clipping is used only to avoid evaluating $\log 0$.

For Pauli rotations the derivative of the model output is computed with the parameter-shift rule,

$$\frac{\partial f(\mathbf{x}; \boldsymbol{\theta})}{\partial \theta_j} = \frac{f(\mathbf{x}; \boldsymbol{\theta}_j^+) - f(\mathbf{x}; \boldsymbol{\theta}_j^-)}{2}, \quad (11)$$

where $\boldsymbol{\theta}_j^{\pm}$ means shifting parameter j by $\pm\pi/2$. Combining Eqs. (10) and (11) gives

$$\frac{\partial L}{\partial \theta_j} = \frac{1}{N} \sum_{i=1}^N \frac{f_i - y_i}{f_i(1 - f_i)} \frac{\partial f_i}{\partial \theta_j}. \quad (12)$$

The reported runs use full-batch Adam. Gradient descent is implemented but was less stable for this small experiment.

D Finite-shot measurement

The main training runs use exact probabilities from the state vector. To mimic a measurement-based circuit run, I also evaluate finite-shot estimates after training. For a sample with exact probability p_i , a shot-based estimate is

$$\hat{p}_i = \frac{k_i}{S}, \quad k_i \sim \text{Binomial}(S, p_i), \quad (13)$$

where S is the number of shots. The finite-shot study repeats this sampling for $S = 100, 1000, 10000$ and reports mean and standard deviation of accuracy and loss. The training is kept exact in this study, so the experiment isolates measurement noise rather than optimizer noise.

III Implementation and validation

A Code structure

The implementation is a Python package in the repository `src/` directory. The central files are:

- `quantum_state.py`: gates, CNOT, state-vector evolution, exact and shot-based measurement;
- `model.py`: encoders, ansaetze, prediction probabilities, binary cross-entropy, and parameter-shift gradients;
- `data.py`: Iris and Breast Cancer loading, stratified splitting, and scaling;
- `optimizers.py`: Adam and gradient descent;
- `baselines.py`: logistic-regression comparison.

The scripts reproduce the main Iris experiment, the circuit-variation study, the Breast Cancer extension, the repeated-seed study, and the finite-shot study. The generated metrics and figures are saved in `results/` and `figures/`.

I used a NumPy state-vector simulator rather than Qiskit or PennyLane in the training code. This keeps the implementation transparent: the submitted code contains the actual gate application, probability evaluation, sampling, loss, and gradient computations. This also makes the parameter-shift rule easy to test against finite differences.

B Data preprocessing and tests

The Iris experiment uses the first two Iris targets, giving a binary problem. The main split is a stratified 75/25 split with random seed 42. Features are scaled to $[0, 1]$ using only the training data, since features are converted into rotation angles. The optional Breast Cancer run uses the first four features and the same scaling procedure.

The test suite checks the most important numerical components. Hadamard and $R_y(\pi)$ tests validate one-qubit gates, a CNOT test validates two-qubit indexing, model tests check prediction shapes and probabilities, the simple ansatz parameter count is verified, parameter-shift gradients are compared with finite differences, and a short two-epoch training smoke test checks that optimization produces finite losses. These tests validate the implementation, although they do not prove that a particular circuit is optimal.

IV Results and discussion

A Basic Iris classifier

The main Iris run used four features, two ansatz layers, the `h_rz` encoder, the simple ansatz, Adam with learning rate 0.05, and 20 epochs. Figure 1 shows that the train loss decreases from about 0.71 to 0.40, while the test loss decreases more slowly and ends near 0.56. This is a useful learning signal, but it also shows a train-test gap.

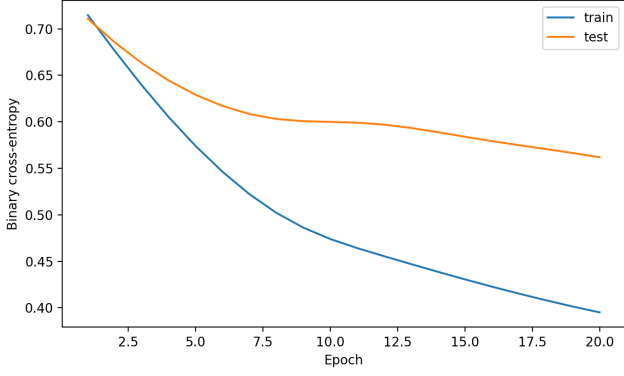


Figure 1: Training and test binary cross-entropy for the basic Iris quantum classifier. The train loss decreases strongly, while the test loss decreases more slowly.

The accuracy curve in Fig. 2 saturates at 0.84 on the test set. Since the test set has 25 points, this corresponds to 21 correct classifications. Logistic regression reaches 1.00 on the same split. Table 1 gives the numerical comparison.

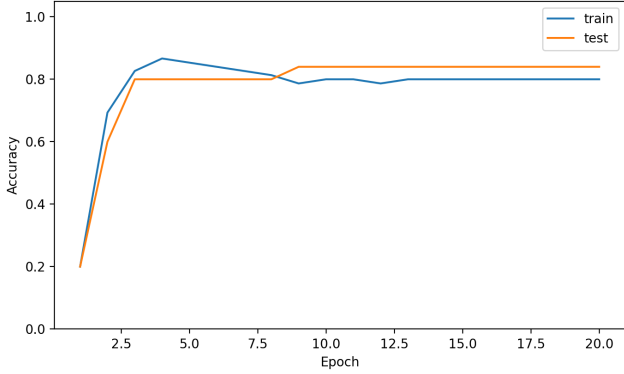


Figure 2: Training and test accuracy for the basic Iris quantum classifier. The final test accuracy is 0.84 for the basic quantum circuit.

Table 1: Main Iris result. Loss is binary cross-entropy. Logistic regression is a classical baseline on the same train/test split.

Model	train acc.	test acc.	test loss
Quantum, h_{rz} /simple	0.800	0.840	0.562
Logistic regression	1.000	1.000	0.132

The confusion matrix in Fig. 3 shows where the errors occur. All 13 class-0 test samples are classified correctly, while four of the 12 class-1 samples are predicted as class 0. Thus the basic circuit does not fail randomly across the two classes; its main weakness is insufficient separation for some class-1 examples.

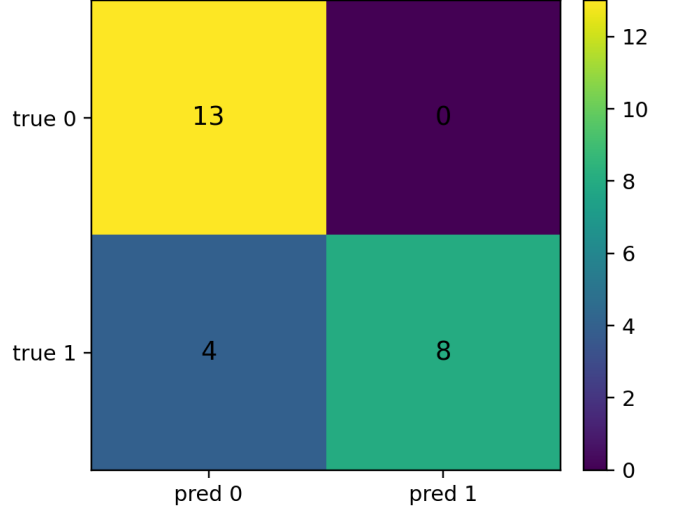


Figure 3: Confusion matrix for the basic Iris quantum classifier. The accuracy is $(13 + 8)/25 = 0.84$.

B Encoder and ansatz variations

The variation study is one of the most important results, because it tests whether the basic circuit is representative. Figure 4 and Table 2 show that it is not. Changing only the encoder from h_{rz} to ry raises the test accuracy from 0.84 to 1.00 on the same split. The ry_{rz} /strong circuit also reaches 1.00 test accuracy, while ry_{rz} /simple and h_{rz} /strong are intermediate.

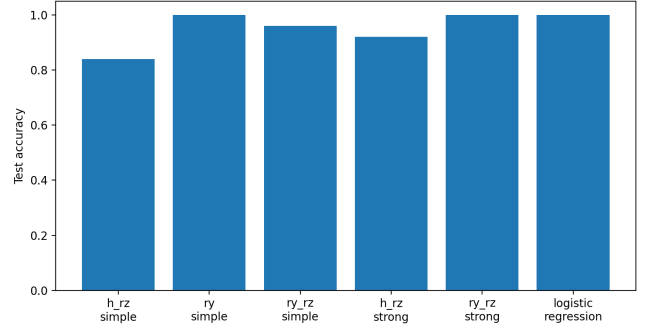


Figure 4: Test accuracy for Iris encoder and ansatz variations. The feature map and ansatz have a large effect on performance.

Table 2: Iris variation study. All quantum models use four features, two layers, 20 epochs, and the same train/test split.

Encoder	Ansatz	N_θ	acc.	loss
h_{rz}	simple	16	0.84	0.56
ry	simple	16	1.00	0.16
ry_{rz}	simple	16	0.96	0.28
h_{rz}	strong	24	0.92	0.39
ry_{rz}	strong	24	1.00	0.31
logistic	regression	–	1.00	0.13

The loss comparison in Fig. 5 adds an important nuance. Even when some quantum circuits reach perfect test accuracy on this small split, logistic regression still has the lowest cross-entropy. The ry encoder gives the best quantum loss. The results therefore support the conclusion that small quantum circuits can learn the binary Iris task, but not that they are better calibrated or more robust than a classical linear baseline.

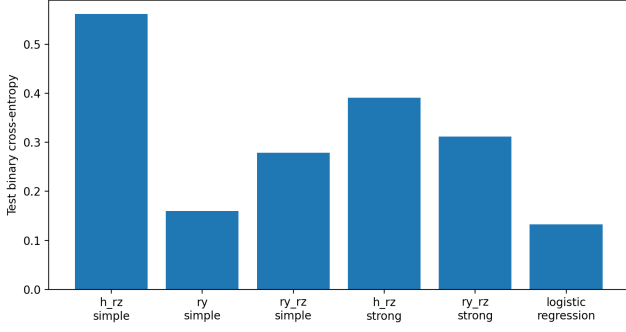


Figure 5: Test cross-entropy for the Iris variation study. The **ry** feature map gives the best quantum-model loss, while logistic regression remains the lowest-loss model.

C Repeated-seed robustness

A single Iris split is not enough to make a strong statistical claim, because one changed test classification changes the accuracy by 0.04. I therefore added a repeated-seed study over several train/test splits and initializations. The results are summarized in Table 3 and Figs. 6–7. The basic **h_rz**/simple circuit remains the weakest model on average, with mean test accuracy about 0.804. The **ry**/simple model is the most robust quantum circuit among the tested choices, with mean test accuracy about 0.992 and mean test loss about 0.143. Logistic regression remains essentially perfect and has the lowest mean loss.

Table 3: Repeated-seed Iris robustness study. Values are mean $\pm$ standard deviation over the generated repeated-seed runs.

Model	test acc.	test loss
h_rz/simple	0.804 ± 0.053	0.433 ± 0.045
ry/simple	0.992 ± 0.019	0.143 ± 0.031
ry_rz/strong	0.956 ± 0.045	0.406 ± 0.035
logistic regression	1.000 ± 0.001	0.126 ± 0.011

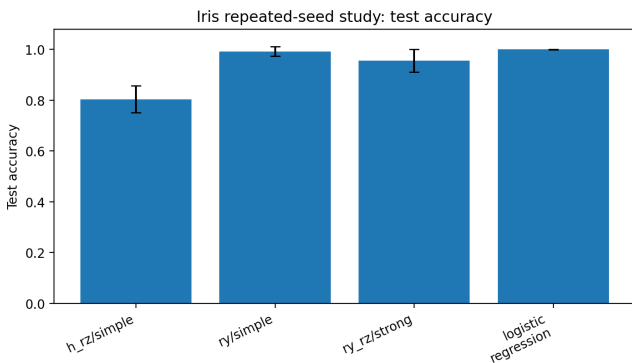


Figure 6: Repeated-seed test accuracy. The **ry**/simple circuit is much more stable than the basic **h_rz**/simple circuit, but logistic regression remains the strongest baseline.

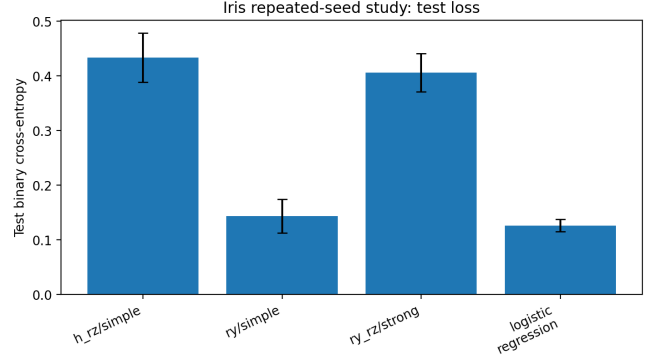


Figure 7: Repeated-seed test loss. The **ry**/simple quantum model has the lowest quantum loss, while logistic regression has the lowest loss overall.

These repeated-seed results strengthen the original variation-study conclusion. The perfect accuracy of a quantum circuit on one split is not by itself strong evidence, but the repeated runs show that the **ry** feature map is consistently better than the original **h_rz** feature map for this implementation.

D Finite-shot measurement

The main results use exact state-vector probabilities. This is useful for debugging and optimization, but the project task also emphasizes measurement. The finite-shot study therefore replaces the final test probabilities by binomial measurement estimates with 100, 1000, and 10000 shots. The training remains exact, so the study isolates measurement noise.

Table 4: Finite-shot Iris measurement study. Values for finite-shot rows are mean $\pm$ standard deviation over repeated measurement simulations.

Model	shots	test acc.	test loss
h_rz/simple	exact	0.840	0.562
	100	0.808 ± 0.037	0.568 ± 0.026
	1000	0.840 ± 0.002	0.563 ± 0.006
	10000	0.840 ± 0.000	0.562 ± 0.001
ry/simple	exact	1.000	0.158
	100	0.994 ± 0.019	0.161 ± 0.007
	1000	1.000 ± 0.001	0.158 ± 0.002
	10000	1.000 ± 0.000	0.158 ± 0.001
ry_rz/strong	exact	1.000	0.311
	100	0.984 ± 0.021	0.316 ± 0.015
	1000	1.000 ± 0.001	0.312 ± 0.004
	10000	1.000 ± 0.000	0.311 ± 0.001

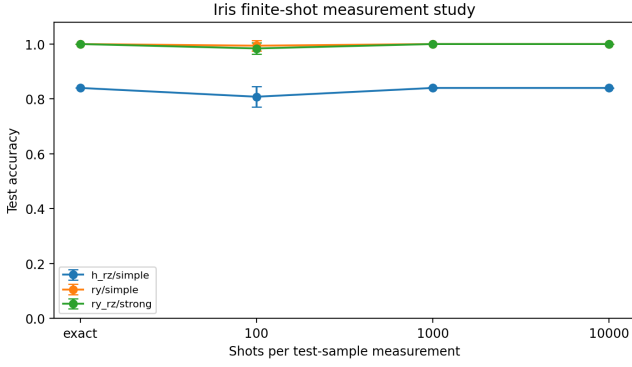


Figure 8: Finite-shot test accuracy for three quantum circuits. With 100 shots, measurement noise affects the weaker basic circuit most strongly; 1000 and 10000 shots reproduce the exact-probability conclusions.

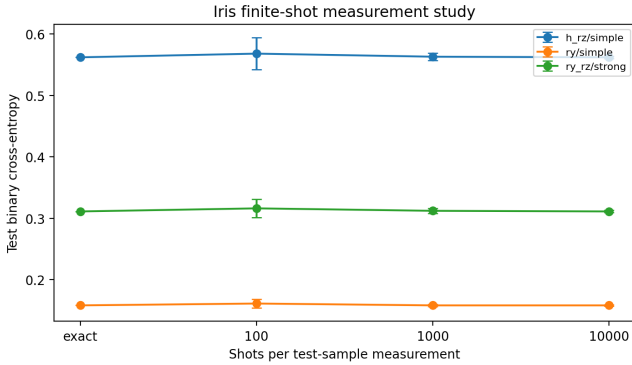


Figure 9: Finite-shot test loss for the same circuits. Shot noise mainly broadens the 100-shot estimates; the ordering of the circuits is unchanged.

The finite-shot results show that the conclusions are not an artifact of exact probability readout. A small number of shots can lower the measured accuracy of borderline circuits, but the better **ry**-based circuits remain stable. This directly connects the state-vector simulation to the measurement interpretation of the project.

E Optional Breast Cancer extension

As an optional extension I used the first four Breast Cancer features with the **ry_rz** encoder and strong ansatz. The training curve in Fig. 10 shows stable learning, and the final quantum test accuracy was 0.874, close to logistic regression at 0.881. Table 5 shows that logistic regression nevertheless has lower loss, indicating better calibrated probabilities.

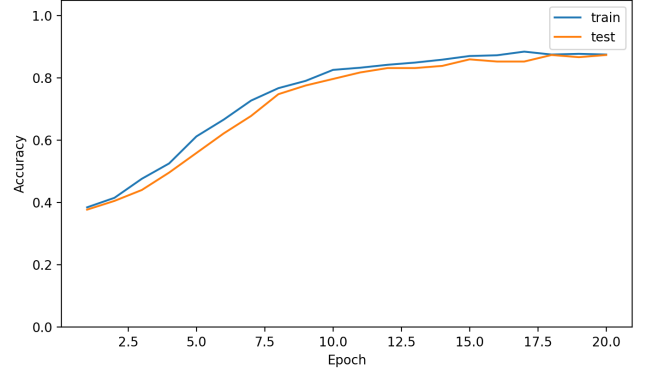


Figure 10: Optional Breast Cancer experiment using the first four features and the **ry_rz/strong** circuit. The quantum model learns a useful classifier and reaches a test accuracy close to logistic regression.

Table 5: Optional Breast Cancer extension. The quantum model is competitive in accuracy, but logistic regression has lower cross-entropy.

Model	train acc.	test acc.	test loss
Quantum, ry_rz/strong	0.876	0.874	0.434
Logistic regression	0.889	0.881	0.296

F Reliability and limitations

The strongest implementation evidence is the agreement between parameter-shift and finite-difference gradients, the gate tests, and the reproducible result files. The repeated-seed and finite-shot studies improve the original analysis by checking two sources of uncertainty: sensitivity to random splits/initializations and measurement noise.

There are still important limitations. The circuits are very small, the Iris data set is simple, and logistic regression is already nearly ideal. The repeated-seed study reduces but does not eliminate statistical uncertainty, and the finite-shot study applies noise only at final measurement, not during training. The Breast Cancer extension uses only four features and accuracy/loss; a more careful medical-classification study should also include balanced accuracy, precision, recall, and F1 score. The correct conclusion is therefore conservative: the quantum models are implemented and can learn, but there is no evidence of quantum advantage here.

V Conclusion

I implemented a complete quantum machine-learning workflow for binary classification: feature encoding, trainable ansaetze, measurement probabilities, binary cross-entropy, parameter-shift gradients, Adam optimization, logistic-regression comparison, circuit variations, repeated-seed robustness, finite-shot measurements, and an optional Breast Cancer extension.

The basic **h_rz/simple** quantum classifier reached 0.84 test accuracy on the binary Iris split, while logistic regression reached 1.00. The variation study showed that circuit design is decisive: the **ry/simple** and **ry_rz/strong** circuits reached 1.00 test accuracy on the same split. The repeated-seed study confirmed that **ry/simple** is more robust than

the basic `h_rz`/simple design, while logistic regression remains the strongest and lowest-loss baseline. Finite-shot measurements did not change the qualitative conclusions; 100 shots introduces visible uncertainty, but 1000 and 10000 shots are close to exact state-vector readout. The Breast Cancer extension showed that the stronger circuit can be competitive in accuracy on a harder data set, but it still did not beat logistic regression in loss.

The main lesson is not quantum advantage. The main lesson is that parameterized quantum classifiers can be built and trained transparently, that the parameter-shift rule gives a practical analytical gradient, and that feature-map and ansatz choices strongly affect performance. Future work should test more random splits, more circuit depths, more classical baselines, noisy training, and data sets where linear decision boundaries are not already sufficient.

Appendix: Use of AI/LLM Tools

ChatGPT was used as a limited supporting tool during this project. The tool was used during parts of the project work in 2026 through the ChatGPT web interface. The implementation, numerical experiments, figures, tables, and final scientific interpretation are my own responsibility.

For the code, ChatGPT was used mainly for support while developing and cleaning the project. This included help with discussing repository organization, interpreting terminal errors, checking Git and $\text{\LaTeX}$ commands, improving file names and structure, and making comments and code organization easier to read. I also used ChatGPT for debugging support and for suggestions about useful consistency checks. All code suggestions were reviewed before use, and the submitted code was tested by running the project workflow and unit tests.

For the written report, ChatGPT was used as a language and feedback tool. It helped with proofreading, improving wording, checking notation consistency, and giving feedback on whether the report addressed the course evaluation criteria. I reviewed and edited the suggestions before including them in the final text.

For text assistance, level 0 means no LLM assistance, level 1 means language-only assistance, level 2 means editorial assistance, and level 3 means generative drafting. In this report the assistance was mainly level 1, with some level 2 editorial feedback for the AI declaration and general report structure. No report section was included without my own review and approval.

For code assistance, level 0 means no LLM assistance, level 1 means debugging or explanation assistance, level 2 means short snippets or small blocks, level 3 means skeleton generation, and level 4 means substantial file-level generation. In this project the assistance was mainly level 1–2: debugging support, command checking, code cleanup suggestions, comments, naming, and short code suggestions. ChatGPT was not used to fabricate numerical results, choose final data, or replace my own calculations and scientific interpretation.

All reported values were generated by running the project code in the GitHub repository. The numerical results were checked against exact diagonalization, analytic limits, unit tests, or other relevant consistency checks.

I take responsibility for the submitted code, figures, tables, numerical values, and scientific interpretation, and I can explain the equations, algorithms, and results used in the final submission.

References

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, Cambridge (2000).
- [2] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, “Evaluating analytic gradients on quantum hardware,” *Physical Review A* **99**, 032331 (2019).
- [3] V. Havlicek *et al.*, “Supervised learning with quantum-enhanced feature spaces,” *Nature* **567**, 209–212 (2019).
- [4] R. A. Fisher, “The use of multiple measurements in taxonomic problems,” *Annals of Eugenics* **7**, 179–188 (1936).
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research* **12**, 2825–2830 (2011).
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” International Conference on Learning Representations (2015).
- [7] Computational Physics, “FYS5419/FYS9419 Project 2: Quantum Computing and Quantum Machine Learning,” University of Oslo, Spring 2026. <https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/Projects/2026/Project2>
- [8] Computational Physics, “Evaluation and grading of projects,” University of Oslo. <https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/Projects/EvaluationGrading>
- [9] Computational Physics, “Project examples and report-writing resources,” University of Oslo. <https://github.com/CompPhysics/QuantumComputingMachineLearning/tree/gh-pages/doc/Projects/ProjectExamples>

Appendix: Use of AI/LLM Tools

ChatGPT was used as a limited supporting tool during this project. The tool was used during parts of the project work in 2026 through the ChatGPT web interface. The implementation, numerical experiments, figures, tables, and final scientific interpretation are my own responsibility.

For the code, ChatGPT was used mainly for support while developing and cleaning the project. This included help with discussing repository organization, interpreting terminal errors, checking Git and LaTeX commands, improving file names and structure, and making comments and code organization easier to read. I also used ChatGPT for debugging support and for suggestions about useful consistency checks. All code suggestions were reviewed before use, and the submitted code was tested by running the project workflow and unit tests.

For the written report, ChatGPT was used as a language and feedback tool. It helped with proofreading, improving wording, checking notation consistency, and giving feedback on whether the report addressed the course evaluation criteria. I reviewed and edited the suggestions before including them in the final text.

Table A1 gives the level of AI assistance used in the written report.

Report section	Level	Description
Abstract	1	Language and clarity check.
Introduction	1	Minor wording and structure feedback.
Theory and methods	1	Language cleanup and notation consistency.
Implementation and validation	1	Wording and readability feedback.
Results and discussion	1	Language cleanup and feedback on clarity of the discussion.
Conclusion	1	Minor wording improvements.
AI declaration	2	Edited to follow the course LLM declaration guidelines.

For text assistance, level 0 means no LLM assistance, level 1 means language-only assistance, level 2 means editorial assistance, and level 3 means generative drafting. In this report the assistance was mainly level 1, with some level 2 editorial feedback for the AI declaration and general report structure. No report section was included without my own review and approval.

Table A2 gives the level of AI assistance used for code and workflow.

Component	Level	Description
Repository organization	1	Suggested cleaner folder structure and Git commands.
Terminal and Git workflow	1	Help interpreting terminal errors and checking commands.
Plotting scripts	1	Help with readability, figure sizing, and report-oriented plotting.
Python source files	1-2	Debugging support, comments, naming, and short code suggestions.
Tests	1-2	Suggestions for useful consistency checks.
LaTeX/report formatting	1	Help with figure placement, formatting, and compilation commands.

For code assistance, level 0 means no LLM assistance, level 1 means debugging or explanation assistance, level 2 means short snippets or small blocks, level 3 means skeleton generation, and level 4 means substantial file-level generation. In this project the assistance was mainly level 1-2: debugging support, command checking, code cleanup suggestions, comments, naming, and short code suggestions. ChatGPT was not used to fabricate

numerical results, choose final data, or replace my own calculations and scientific interpretation.

All reported values were generated by running the project code in the GitHub repository. The numerical results were checked against exact diagonalization, analytic limits, unit tests, or other relevant consistency checks. I take responsibility for the submitted code, figures, tables, numerical values, and scientific interpretation, and I can explain the equations, algorithms, and results used in the final submission.